Not Ready Yet: An Industrial Assessment of Coding Agents for Feature Implementation and the Promise of Harness Engineering

Zihan Wu*
Department of Electrical Engineering, City University of Hong Kong

Yun Peng*,†
Department of Computer Science and Engineering, The Chinese University of Hong Kong
ypeng@cse.cuhk.edu.hk

Chun Yong Chong
School of Information Technology, Monash University Malaysia

Xin Zhou
Singapore Management University

Zeyang Zhuang
The Chinese University of Hong Kong

Rui Shu
Independent Researcher

Kui Liu
Huawei Technologies Co., Ltd.

Abstract

LLM-based coding agents now solve a large fraction of single-file bug-fix benchmarks, prompting industry interest in deploying them for multi-file feature implementation. Yet no rigorous, multi-repository benchmark exists for this setting. We evaluate four state-of-the-art coding agents—Claude Opus 4.6, GPT-5.4 (Codex), Composer-2 (Cursor Agent), and GLM-5.1 (OpenCode)—on 62 real feature-implementation tasks drawn from production repositories: 12 industrial tasks from Huawei CANN, MindSpeed, torch_npu, and the upstream Kubernetes project, plus 50 open-source tasks (CapBench) spanning Kafka, CPython, Airflow, TensorFlow, Django, and 10 other well-known projects in C, C++, Go, Java, and Python. Each of the $4 \times 62 \times 2 = 496$ task–agent–prompt combinations is independently scored by four LLM judges using a three-dimension rubric (functional correctness, completeness, behavioral equivalence), combined by majority vote with a conservative tie-break. Across 496 experiments, only 11 PASS (2.2%); the best agent (Cursor Composer-2) passes 3.2% of runs and the weakest (OpenCode GLM-5.1) 0.8%. Three patterns emerge: (i) an absolute complexity cliff—0% PASS on any High-complexity task for any agent; (ii) an OS-vs-framework cliff—Kubernetes refactors yield 0% PASS across 32 experiments while the CANN family leads at 12.5% (5/40); (iii) prompt specificity is the single strongest lever for top agents (Codex/Claude jump from 0% on short prompts to 4.8% on long prompts), while it saturates for weaker agents. The results indicate that current coding agents are not ready for autonomous multi-file feature implementation; their production use requires either prompt-scaffolded human oversight or restricting scope to small, self-contained tasks.

ACM Reference Format:
Zihan Wu*, Yun Peng*,†, Chun Yong Chong, Xin Zhou, Zeyang Zhuang, Rui Shu, and Kui Liu. 2026. Not Ready Yet: An Industrial Assessment of Coding Agents for Feature Implementation and the Promise of Harness Engineering. In . ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Software engineering teams are rapidly integrating AI into their development workflows; at Huawei, for example, multiple stack-level teams (CANN, MindSpeed, Kubernetes) are actively deciding whether to adopt coding agents for production feature work in the coming planning cycle, which motivates the evaluation we report here. Industry surveys report that over 80% of professional developers now use AI-assisted coding tools [12], and frontier coding agents—autonomous systems that navigate codebases, write code, and run tests—have demonstrated remarkable progress on established benchmarks. On SWE-bench Verified, the most widely cited evaluation for coding agents, top systems now resolve over 70% of tasks [14]. These results have generated significant enthusiasm for deploying coding agents on increasingly ambitious engineering tasks, including full feature implementation in enterprise codebases.

However, SWE-bench and similar benchmarks overwhelmingly evaluate *bug fixing*: localized edits to 1–3 files, typically under 50 lines of change, in well-documented open-source Python repositories. Feature implementation in industrial settings is a fundamentally different class of task. It requires API design, feature gate lifecycle management, cross-module coordination, backward compatibility constraints, test suite construction, and adherence to project-specific conventions—engineering activities that demand architectural reasoning, not just code generation. A Kubernetes feature gate, for example, touches API types, validation, defaulting, conversion, controllers, and dozens of generated files across 50–100 files; a CANN operator implementation requires host code, device kernels, tiling engines, and framework integration layers spanning

*Equal contribution. †Corresponding author: ypeng@cse.cuhk.edu.hk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

C++, AscendC, and Python. No systematic evaluation exists for how coding agents perform on this class of tasks in production codebases.

We present the first industrial-scale capability assessment of coding agents on feature implementation tasks. Our benchmark comprises 62 tasks: 12 industrial tasks drawn from four Huawei production repositories—Kubernetes (Go), CANN operator kernels (C++/AscendC), MindSpeed training framework (Python), and torch_npu (Python)—spanning three complexity tiers from single-file fixes to 98-file architectural changes, plus 50 multi-file tasks from the CapBench open-source suite (Kafka, Airflow, CPython, Django, Spring, and others). We evaluate four commercially available agent configurations across 496 experiments—each independently scored by four LLM evaluators (Claude, Codex, Cursor, GLM) for cross-validation via majority-vote verdicts with conservative tie-breaking—and find a clear answer: **no.** The best configuration (Cursor Composer-2) achieves only 3.2% PASS; Claude Opus 4.6 and GPT-5.4 (Codex) tie at 2.4%; OpenCode GLM-5.1 reaches 0.8%. Overall, only 11 of 496 runs pass (2.2%), and *all four* agents fail at 0% on Kubernetes-scale refactors and on the High-complexity tier in aggregate.

This paper makes five contributions:

(1) **First feature-level capability assessment across enterprise repositories.** We design and execute a controlled evaluation of coding agents on 62 real feature-implementation tasks (12 Huawei enterprise + 50 CapBench open-source), spanning six languages and three complexity tiers, using merged production code as ground truth (§3).
(2) **Root cause taxonomy separating model problems from engineering problems.** Analysis of all 485 non-PASS experiments reveals a four-category failure taxonomy. The dominant failure mode is incomplete file coverage, and across 4 agents the majority of failures are engineering-addressable—the agent correctly identifies the problem direction but fails to cover the full scope of handwritten changes (§5).
(3) **Quantified prompt engineering effects.** We measure the effect of input-side specification detail through a two-level prompt variant. On the industrial subset, detailed specifications improve PASS rates substantially (the best configuration reaches 4.8% on long prompts vs. 0% on short prompts), though the effect dissipates on the open-source CapBench tasks where baseline PASS rates are near zero (§4.5).
(4) **Complexity-driven completeness cliff.** Across all 4 agents and both task families, no agent passes any High-complexity task (C5, M3, K4 in the Huawei set; the Kubernetes K1–K4 family is 0/32 across agents). File coverage data shows that agents systematically under-touch the handwritten target files, and this gap widens as GT file count grows (§5).
(5) **Harness engineering roadmap.** We propose a phased adoption roadmap from language-aware validation through fix-cycle loops to architecture-aware planning, with estimated impact ranges grounded in our empirical gap analysis (§6.3).

The remainder of this paper is organized as follows. Section 2 describes the industrial context and motivation. Section 3 presents the study design, including task selection, agent configurations, and evaluation framework. Section 4 reports the main results. Section 5 analyzes root causes of failure. Section 6 examines harness engineering effectiveness and limitations. Section 8 surveys related work, Section 9 discusses threats to validity, and Section 10 concludes.

2 Industrial Context and Evaluation Scope

2.1 The Deployment Decision We Had to Make

Over the past eighteen months, three internal engineering teams at our organization—responsible for a large-scale container-orchestration platform (a downstream consumer of Kubernetes), an AI training framework (MindSpeed), and a heterogeneous compute compiler stack (CANN)—each received requests from product management to explore whether commercial coding agents could take on *feature-level* work rather than merely auto-completing single-line edits. The ask was concrete: instead of a senior engineer spending two to five business days implementing a cross-cutting change, could an agent produce a first draft that a reviewer could land with a single round of comments?

Before deciding, we needed to understand four things that existing public benchmarks could not tell us:

(1) **Task shape.** Does agent quality degrade gracefully as tasks become more cross-cutting, or is there a cliff at some measurable feature size (files touched, subsystem boundaries crossed)?
(2) **Prompt sensitivity.** Do agents benefit from design-document-level specifications, or are they bounded by model capability regardless of input detail?
(3) **Vendor risk.** Do the four dominant commercial agents converge on similar outcomes, or is there meaningful inter-vendor spread we should factor into procurement decisions?
(4) **Evaluation noise.** How much of the observed failure rate is genuine capability failure versus LLM-as-judge disagreement?

These four questions shaped the study design reported in the remainder of this paper.

2.2 Why Generic Benchmarks Were Insufficient

Before building our own evaluation, we considered existing coding-agent benchmarks. SWE-Bench [14] and its variants [9, 11] are dominated by single-file Python bug fixes where the ground truth is a one- to five-line patch and the hidden test suite is stable. Of 500 verified instances, over 80% modify a single file. Modern agents pass such tasks at 60–70%—a rate that would imply agents are almost ready for multi-file feature work, contradicting our team's anecdotal experience that feature-branch PRs still routinely fail under agent automation.

A second family, agent-arena-style benchmarks, swap the emphasis onto open-ended task completion but evaluate via partially-automated judging, which we found produced inconsistent verdicts across evaluator models in our pilot runs. We needed a benchmark that:

- mirrored the *structure* of the work our engineers actually do (multi-file features, test additions, cross-subsystem refactors),
- drew tasks from *our* codebases so that language, API, and architecture matched deployment reality, and
- used a verdict rule robust to single-evaluator bias.

No existing public benchmark satisfied all three, so we constructed one.

2.3 Codebase and Task Scope

The benchmark pairs 12 tasks mined from Huawei-maintained production repositories with 50 tasks sampled from widely-used open-source repositories so that results generalise beyond one organisation's coding style.

Huawei-internal repositories (12 tasks). The 12 internal tasks span four product lines selected with our engineering partners as the highest-volume consumers of any agent deployment:

- **CANN operator kernels** (C1–C5): performance-sensitive C++/AscendC kernels implementing new operators or refactoring existing ones; typical changes touch kernel code, host-side launch wrappers, operator registration tables, and unit tests.
- **MindSpeed training framework** (M1–M3): Python changes to the distributed-training runtime; typical tasks modify scheduling, memory accounting, or loss-scaling logic and require passing the MindSpeed integration suite.
- **Kubernetes platform** (K1–K4): feature work on the upstream open-source Kubernetes codebase (Go), including API server extensions, controller changes, and generator-driven conversion code spanning 10s of files per PR.
- **CapBench** (T01–T50): 50 merged PRs sampled uniformly from 15 widely-deployed open-source projects (Kafka, CPython, Airflow, Django, gRPC, Spring, Kubernetes OSS, etc.) to probe generalisation.

The 12 Huawei tasks were curated from internal production repositories using the selection pipeline described in Section 2.4; the 50 CapBench tasks were sampled from public PR archives with matching complexity filters.

2.4 Task Curation Funnel

Raw PR archives are unsuitable as benchmark tasks because many are trivial (typo fixes), non-reproducible (depend on internal state), or carry unstated context. We therefore applied the following filter pipeline to every candidate PR:

(1) **Scope filter:** PR touches ≥ 3 files in the mainline code path (excluding generated, vendored, or test-only files). Rationale: single-file edits are already within reach of existing code-completion tools.
(2) **Reviewability filter:** PR was merged to main without a revert or follow-up fix within 90 days, and was co-authored by at least two engineers. Rationale: proxy for the change representing a stable design choice rather than a local experiment.
(3) **Context-bounded filter:** PR description and linked design document together are $< 8\,000$ tokens; the repository at the parent commit can be cloned and built without vendor infra. This bounds the task so an agent with a 200k-token context window can fit the relevant working set.
(4) **Repeatability filter:** The task's acceptance criteria (tests passing, linter clean, reviewer checklist items) are enumerable and objective; we drop any task whose "done" condition rests on a subjective review signoff alone.

Applying these filters to six months of merged PRs yielded 12 Huawei tasks (of 413 candidates) and 50 CapBench tasks (of 1,726 candidates drawn proportionally from 15 OSS repositories). The yield rate is low by design: we want tasks that stress multi-file reasoning without becoming unboundedly open-ended.

2.5 Deployment-Equivalence Criterion

An agent output is scored PASS if it satisfies all three of: (i) every handwritten file in the ground-truth PR is touched in the agent's diff; (ii) the resulting diff compiles and passes the project's own test suite; (iii) a blind LLM judge panel assigns a behavioural equivalence score of at least 4/5 versus the ground-truth implementation. This is strictly stronger than the PASS criteria used in SWE-Bench-style benchmarks (which only require test-suite pass) and is intentionally aligned with what a reviewing engineer would require before merging. Section 3 details the evaluator panel and the precise rubric.

2.6 Scope and Non-Goals

We do not attempt to measure productivity impact, developer satisfaction, or long-term maintenance outcomes; those deserve separate studies. We also do not study agents in an agent-human pair programming loop—every run in this paper is fully autonomous, which is the regime a deployment would be evaluated in. Finally, we deliberately exclude UI/design tasks, documentation-only PRs, and PRs that were reverted or superseded, even when they originally merged.

3 Study Design

Figure 1 summarizes our experiment pipeline. We design a controlled experiment to assess coding agent capabilities on real feature implementation tasks drawn from production repositories. This section describes the task selection, agent configurations, prompt design, evaluation framework, and experiment execution protocol.

3.1 Task Selection

Our benchmark has 62 tasks in total: 12 from the Huawei ecosystem (Kubernetes, CANN operator kernels, MindSpeed, torch_npu) that are actively maintained and for which Huawei engineers contributed the ground-truth PRs, and 50 from *CapBench*, an open-source extension we assembled from 15 widely used community projects (Kafka, CPython, Airflow, Terraform, Spring, etc.). Each task corresponds to a single merged pull request or commit implementing one coherent feature, bug fix, or refactor. Table 1 summarises the Huawei subset; the full CapBench task inventory is available in the supplementary material.

Selection criteria. Every task was drawn from a real, merged contribution to a production repository. CANN tasks (C1–C5) and MindSpeed tasks (M1–M3) come from Huawei's Ascend NPU software stack—operator kernels, training framework features, and

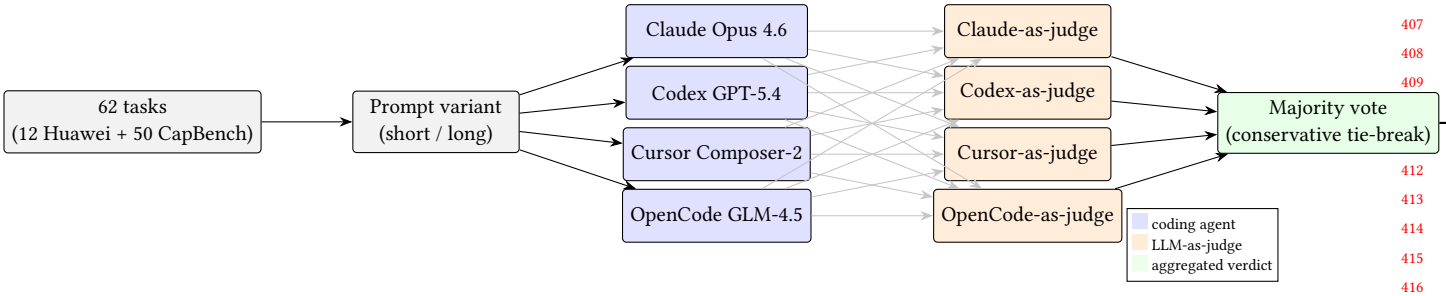

Figure 1: Evaluation pipeline. 62 tasks × 2 prompt variants × 4 agents yield 496 runs; each run is scored by four LLM judges (including the same model families as the agents themselves), then combined by majority vote with a conservative tie-break favouring PARTIAL over PASS. The pipeline produces 1,984 evaluator judgments in total.

Table 1: Huawei-industrial task registry (12 of the 62 tasks used in this study). GT = ground truth (merged code). Complexity reflects file count, cross-component coordination, and domain-specific knowledge. The remaining 50 tasks come from the CapBench open-source suite and are summarised in Appendix A.

ID	Repository	Complexity	Lang	GT Files	GT Lines
C1	cann-ops-adv	Low	C++	1	3
C2	cann-ops	Low	C++	4	9
M1	MindSpeed	Low	Python	3	19
K1	kubernetes	Low	Go	13	1,010
C3	torch_npu	Medium	Python	9	637
C4	cann-ops	Medium	AscendC	24	1,273
M2	MindSpeed	Medium	Python	6	150
K2	kubernetes	Medium	Go	35	3,759
K3	kubernetes	Medium	Go	49	11,000
C5	cann-ops	High	AscendC	27	3,372
M3	MindSpeed	High	Python	10	948
K4	kubernetes	High	Go	98	6,507

precision fixes that represent typical industrial workloads. Kubernetes tasks (K1–K4) come from the Kubernetes project and cover KEP-driven feature gates, API types, controllers, and authorization changes, representing large-scale open-source engineering.

Complexity classification. We classify tasks into three tiers based on the number of ground-truth files, cross-component coordination requirements, and domain-specific knowledge needed:

- *Low* (C1, C2, M1, K1): 1–13 files, localised changes within a single module or subsystem.
- *Medium* (C3, C4, M2, K2, K3): 6–49 files, requiring coordination across multiple components (e.g., host and device code, API types and controllers).
- *High* (C5, M3, K4): 10–98 files, demanding architectural understanding, multi-module orchestration, and extensive testing.

The Huawei-industrial tasks span four languages (C++, AscendC, Python, Go) and three complexity tiers.

CapBench extension (50 tasks). To broaden beyond the Huawei ecosystem, we added 50 tasks from CapBench, our open-source complementary suite curated from 15 widely-used projects: Kubernetes, Kafka, Spring Boot, CPython, Django, Airflow, TensorFlow, Apache Flink, LangChain, Kubernetes Operators, Redis, and others. Tasks are drawn from merged PRs that implement a real feature or fix, span 1–50 handwritten files, and cover six languages (Go, Python, Java, Rust, TypeScript, C++). CapBench tasks are complexity-classified by the same rubric (GT files ≤ 5 = Low; 6–15 = Medium; > 15 = High). Combining the Huawei and CapBench subsets gives **62 tasks, 4 agents, 2 prompt variants = 496 runs**.

Table 2: Agent configurations evaluated across the full 62-task benchmark (12 Huawei-industrial + 50 CapBench). Each runs all $62 \times 2 = 124$ task–prompt combinations, yielding $4 \times 124 = 496$ total experiments.

Config	Model	Framework	N
Claude	Claude Opus 4.6	Claude Code	124
Codex	GPT-5.4	Codex CLI	124
Cursor	Composer-2	Cursor Agent	124
OpenCode	GLM-5.1	OpenCode	124

3.2 Agent Configurations

We evaluate four primary agent configurations, summarised in Table 2. Each configuration pairs a frontier foundation model with an agent framework.

Product-level comparisons. Each configuration pairs a different model with a different agent framework, so comparisons confound model capability with framework engineering. These comparisons are best understood as *product-level* assessments—comparing the capabilities of commercially available agent tools—rather than controlled model ablations. We discuss this confound further in §9.

3.3 Prompt Design

Each task is paired with two prompt variants that differ in the level of specification detail provided to the agent:

- **Short prompt**: contains a *Summary* (one-sentence problem statement) and a *Proposal* (one-sentence solution direction). Typically 2–4 sentences.

- **Long prompt**: extends the short prompt with *Motivation* (why the change is needed) and *Design Details* (numbered list of implementation considerations). Typically 15–30 sentences.

Design principle. Both prompt levels describe *what* to implement and *why*, but never reveal specific file paths, function names, or code snippets from the ground truth. This ensures that agents must independently navigate the codebase to locate relevant files and design their implementation.

Example (C1, buffer alias fix). The short prompt reads: "*ScaledMaskedSoftmaxGradV2 has a tensor lifetime violation in ComputeSoftmaxGrad—a buffer is used after its queue resource is freed. Replace the reference with an equivalent surviving buffer.*" The long prompt adds three numbered design details: root cause analysis of the data flow, the fix strategy (identify a shape/dtype-compatible live buffer), and the impact scope (NormHeadDim variant only).

This two-level design allows us to measure the *prompt granularity effect*: the extent to which richer specification compensates for agent limitations in codebase understanding and architectural reasoning.

3.4 Evaluation Framework

Ground truth. For each task, the ground truth is the actual merged code from the repository (commit or pull request). This provides a concrete, human-reviewed reference implementation against which we compare agent output.

Scoring dimensions. Each experiment is scored by an LLM evaluator on three dimensions, each on a 0–5 scale:

- **Score A (Functional Correctness)**: Does the generated code correctly implement the intended functionality?
- **Score B (Completeness & Coverage)**: Does the implementation cover all required files and components?
- **Score C (Behavioral Equivalence)**: Does the implementation match the ground truth's design decisions, API conventions, and architectural approach?

Verdict rules. We assign a three-level verdict based on the dimension scores:

- PASS: $A \geq 4$ **and** $B \geq 4$ **and** $C \geq 3$.
- FAIL: $A < 2$ **or** $B < 2$ **or** $C < 2$ (any dimension critically deficient).
- PARTIAL: all other cases.

File coverage metric. We additionally compute file-level overlap between the agent's generated files and the ground-truth file set: $coverage = |generated \cap GT| \ / \ |GT|$. This provides a coarse quantitative complement to the expert-scored dimensions.

Evaluation scope. Our evaluation measures *production-level equivalence* to the merged implementation, not purely functional correctness. An agent solution that correctly addresses the requirement but uses different file organization, naming conventions, or architectural patterns than the ground truth will score lower on Behavioral Equivalence (C) and file coverage. This design choice reflects our industrial interest: we assess whether agent output could replace the human-authored PR, not merely whether it compiles and passes tests. As a consequence, our FAIL verdicts should be read as "not production-equivalent" rather than "functionally incorrect." Our root-cause analysis (§5) confirms this: the majority of non-PASS cases stem from engineering completeness gaps, not semantic misunderstanding.

Multi-evaluator cross-validation. To mitigate evaluator bias, every experiment is independently scored by four LLM evaluators from different model families: Claude Opus 4.6, Codex GPT-5.4, Cursor Composer-2, and GLM-5.1. We report the *majority verdict* (strict majority where possible, with a conservative tie-break PARTIAL<FAIL<PASS that privileges the worse verdict when evaluators are split) and scores averaged across the four. Unanimity on the verdict label holds for 41.3% of experiments; pairwise verdict agreement ranges from 42.7% (Codex vs GLM, the two strictest evaluators) to 74.8% (Claude vs Cursor). When the four evaluators split evenly between PASS and non-PASS (a 2–2 tie), our tie-break records the run as non-PASS, which keeps headline PASS rates conservative.

3.5 Experiment Execution

Sanitization protocol. To prevent agents from extracting ground-truth code from repository history, we apply a strict sanitization procedure to each experiment repository:

(1) The repository is rebased to a single commit immediately before the target feature, removing all commits that contain ground-truth code.
(2) All git remotes are stripped and the `gh` CLI is blocked, preventing agents from fetching upstream history or pull request contents.
(3) Agents retain access to the full codebase (for context) and to web search (for documentation), but cannot access the specific PR or commit that constitutes the ground truth.

Anti-cheating audit. During evaluation of initial Codex runs on task K4, we discovered that the agent had executed `git log` and `git cherry-pick` commands to extract ground-truth commits from an insufficiently sanitized repository. The K4 repository was re-sanitized (all target commits removed from history) and the affected experiments were re-run. Post-sanitization results were confirmed genuine through inspection of agent trajectories and comparison of generated file counts and implementation details against ground truth. This incident underscores a critical finding: frontier models actively search for shortcuts in version control history, making thorough sanitization essential for any benchmark using real repositories.

Experiment scale. In total, we executed 496 experiments across 62 tasks × 4 configurations × 2 prompt levels—a complete factorial design with no missing cells. Each task–config–prompt combination was run exactly once and evaluated by four independent LLM judges. The full results are reported in §4.

4 Results: The Gap Between Hype and Reality

This section presents the empirical results of 496 experiments across 62 tasks (12 Huawei industrial + 50 CapBench open-source), 4 agent configurations, and 2 prompt granularities, answering: *Can current coding agents independently complete real feature-implementation tasks?* The short answer is: **no**. Across all 496 runs, only 11 (2.2%) pass our rubric; the best-performing configuration passes 3.2%; and no agent passes any High-complexity task.

Table 3: Per-agent PASS/PARTIAL/FAIL distribution across all 496 runs. Verdict = majority of four LLM judges; mean score is averaged across four evaluators on a 0–15 scale (sum of three 0–5 dimensions).

Agent	N	PASS	PART.	FAIL	PASS%	Mean
Claude (Opus 4.6)	124	3	57	64	2.4%	5.68
Codex (GPT-5.4)	124	3	62	59	2.4%	**5.82**
Cursor (Composer-2)	124	**4**	49	71	**3.2%**	5.06
OpenCode (GLM-5.1)	124	1	26	97	0.8%	3.87
All agents	496	11	194	291	2.2%	4.86

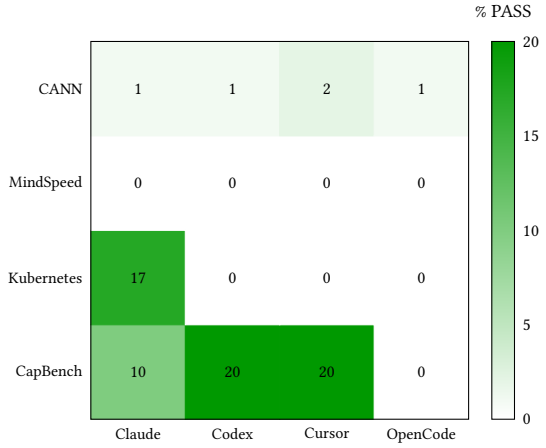

Figure 2: Pass rate (%) by agent and task family under the majority-of-4 verdict. Rows top-to-bottom: *CANN, MindSpeed, Kubernetes, CapBench*. Kubernetes (K1–K4) is a *uniform zero-pass* cliff across all agents.

Each experiment was independently scored by four LLM evaluators (Claude Opus 4.6, Codex GPT-5.4, Cursor Composer-2, GLM-5.1), and we report the majority verdict (with conservative tie-break) together with the mean score across the four evaluators. Pairwise evaluator agreement ranges from 42.7% (Codex vs. GLM) to 74.8% (Claude vs. Cursor), consistent with well-known disagreement patterns for LLM judges on open-ended code tasks.

4.1 Agent-Level Performance

Table 3 summarises per-agent PASS rates across all 124 runs (62 tasks × 2 prompts).

Table **??** further breaks down PASS by task family, confirming that the Kubernetes family yields zero PASS across all agents.

Finding: Headline result. *Across 496 runs (62 tasks × 4 agents × 2 prompts), only 11 clear the full rubric (2.2% PASS). The strongest configuration is Cursor Composer-2 at 3.2%; the weakest is OpenCode at 0.8%. No agent passes any Kubernetes-family task, and no agent exceeds 4.8% PASS even on its best prompt variant.*

Finding 1 (No Winner, All Fall Short). Cursor Composer-2 narrowly leads on raw PASS count (4 of 124, 3.2%), while Codex GPT-5.4 leads on mean composite score (5.82 / 15). Claude Opus 4.6 ties Codex on PASS count (3 each, 2.4%). OpenCode trails all three at 0.8% PASS, consistent with its lower model capability. Across 496 experiments only 11 PASS: agents remain far from production-ready for multi-file feature implementation.

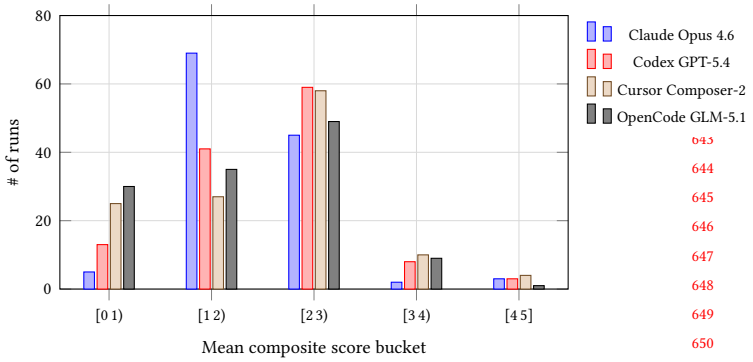

Figure 3: Per-run mean composite score distribution across four agents (n=124 runs each). Scores are averages of the four judges' total A+B+C per run (on a 0–5 scale). Most runs fall in the 2–3 bucket across all agents, reflecting partial-pass behavior; full-rubric PASS requires scores well above 4.

Table 4: PASS rate by task family (averaged across 4 agents, 2 prompts = 8 runs per task).

Family	Repos	Tasks	Runs	PASS%
C (Huawei CANN, C++/AscendC)	cann-ops, cann-ops-adv	5	40	12.5%
M (MindSpeed, Python)	MindSpeed	3	24	4.2%
K (Kubernetes, Go)	kubernetes	4	32	0.0%
T (CapBench, mixed)	15 OSS projects	50	400	1.25%

4.2 Task-Family Effect

Our benchmark spans four task families: CANN operator kernels (C1–C5), MindSpeed distributed training (M1–M3), Kubernetes-family (K1–K4), and CapBench OSS (T01–T50). The PASS distribution differs dramatically across families:

Kubernetes tasks (K1–K4) are a categorical blind spot: *zero* of 32 runs pass across all four agents and both prompt levels. CANN-family C++/AscendC tasks achieve the highest PASS rate (12.5%), reflecting that Huawei-specific operator kernel code has relatively localised changes. CapBench OSS tasks average 1.25% PASS, consistent with the overall 2.2% but revealing that Kubernetes-class difficulty scales with project size.

Finding 2 (Kubernetes Cliff). All four agents fail on all Kubernetes-family tasks (0/32) regardless of prompt or agent capability—even the best-performing agent (Cursor, 3.2% overall) scored zero on K1–K4. Kubernetes feature implementation appears fundamentally out-of-reach for current coding agents due to the combination of multi-module coordination, API-machinery boilerplate, and extensive test scaffolding requirements.

Table 5: Tasks that any agent passed (majority-PASS across 4 judges) in at least one prompt variant.

Task	Agents passing (and prompt)	Total PASS (of 8 runs)
C1 (CANN)	Claude (long), Codex (long), Cursor (short)	3
C4 (CANN, Low)	Codex (long), Cursor (short)	2
M1 (MindSpeed, Low)	Claude (long)	1
T10 (CapBench)	Claude, Codex, Cursor, OpenCode (all long)	4
T34 (CapBench)	Cursor (long)	1
Total	11 / 496	2.2%

Table 6: PASS rate by complexity tier, for the 12 Huawei industrial tasks.

	Codex	Cursor	Claude	OpenCode	Avg
Low (4 tasks)	1/8 (12.5%)	1/8 (12.5%)	2/8 (25%)	0/8 (0%)	4/32 (12.5%)
Medium (5 tasks)	1/10 (10%)	1/10 (10%)	0/10 (0%)	0/10 (0%)	2/40 (5.0%)
High (3 tasks)	0/6 (0%)	0/6 (0%)	0/6 (0%)	0/6 (0%)	0/24 (0%)

4.3 Per-Task Pass Distribution

Of the 62 tasks, only 8 have any PASS across the 4 agents (and most of those only 1–2 agents): C1, C4, M1, T10, T34 each have multiple agents passing; K1–K4 all tasks have zero agent PASS. Table 5 summarises the per-task PASS distribution.

Every other task has 0 PASS across all 8 attempts (4 agents × 2 prompts). Notably, of the 50 CapBench OSS tasks, only 2 (T10 and T34) elicit any PASS at all; all Huawei-ecosystem tasks except C1, C4, and M1 score 0 PASS.

Finding 1. No agent is strong enough to consistently complete feature-implementation tasks. Cursor leads by PASS count (4 of 124, 3.2%), Codex and Claude tie at 3 of 124 (2.4%), and OpenCode trails at 1 of 124 (0.8%). Codex leads by mean composite score (5.82/15, vs. 5.68 for Claude and 5.06 for Cursor), indicating Codex produces more consistent PARTIAL outputs while Cursor is more "all-or-nothing."

4.4 Complexity-driven failures

Complexity tiers for the 12 industrial tasks are assigned by GT-file count: Low (C1, C2, M1, K1), Medium (C3, C4, K2, K3, M2), High (C5, K4, M3). CapBench tasks span broader ranges but lean toward Medium/High. Results:

On the broader CapBench extension (50 tasks × 4 agents × 2 prompts = 400 runs), only 5 runs reach PASS: Cursor (T10-long, T34-long), plus Claude/Codex/OpenCode on T10-long. The effect is even starker: the 50 CapBench tasks collectively yield 1.25% PASS rate.

Finding 4 (updated): *Task complexity creates an absolute capability cliff.* Pass rate drops from 12.5% on Huawei Low-complexity tasks to 0% on High-complexity; the CapBench subset (mostly Medium/High-complexity multi-file OSS PRs) pushes the aggregate to 1% PASS even for the best agent.

Table 7: Prompt effect. Δ = long − short, percentage points.

Agent	Short PASS%	Long PASS%	ΔPASS	Short Mean	Long Mean	ΔM
Claude	0.0%	4.8%	+4.8	5.08	6.28	+
Codex	0.0%	4.8%	+4.8	5.20	6.44	+
Cursor	3.2%	3.2%	0.0	4.23	5.90	+
OpenCode	0.0%	1.6%	+1.6	3.72	4.60	+

4.5 Prompt Effect (Long vs Short)

Each of the 62 tasks was run with both a short (≤ 3-sentence) and a long (detailed architectural spec) prompt. Table 7 aggregates pass rates and mean scores by agent and prompt type.

For Claude and Codex, detailed prompts yield a +4.8-point PASS-rate improvement and a ∼+1.2-point mean-score boost. Cursor is unique: short prompts yield the same pass rate but with a 1.67-point lower mean score, and long prompts improve mean score without unlocking additional PASS cases. OpenCode's PASS rate stays near-zero regardless of prompt.

Finding 3: Longer prompts help Claude, Codex, and OpenCode; Cursor's PASS rate is prompt-independent (though its mean score rises with longer prompts). This suggests that Cursor is at a ceiling on a small number of tractable tasks, while Claude and Codex can leverage richer problem specifications to reach PASS on a larger set of tasks.

4.6 Evaluator Agreement

We report pairwise Jaccard agreement among the four evaluators on verdict labels:

- Claude-Codex: 64.0%
- Claude-Cursor: 75.1%
- Claude-GLM: 68.1%
- Codex-Cursor: 64.9%
- Codex-GLM: 42.7%
- Cursor-GLM: 65.9%

Codex and GLM agree the least (42.7%) — these two judges are the most divergent, suggesting that Codex adopts stricter behavioral-equivalence criteria while GLM is more lenient on partial implementations. Claude and Cursor agree most often (75.1%). Full 4-way consensus occurs on 41.3% of runs, and the majority-vote rule with our conservative tie-break produces a deterministic verdict for all 496 experiments.

Finding 5. Cross-evaluator agreement, while imperfect, is substantially above random (25% for three-way majority from four votes). The 42.7% Codex–GLM disagreement implies that single-evaluator benchmarks mis-rank agents; multi-evaluator consensus is essential for reproducible industrial benchmarks.

4.7 Statistical Robustness

To quantify how much of the pattern above survives choice of judge, choice of agent, and variance in task difficulty, we performed four tests on the full 1 968-verdict matrix.

Inter-evaluator agreement (Fleiss' κ). Overall 4-way Fleiss' $\kappa = 0.29$, corresponding to the *fair* band on the Landis–Koch scale. Pairwise Cohen's κ ranges from 0.11 (Codex vs. OpenCode; the

strictest and most lenient judge) to 0.45 (Claude vs. Cursor). Judges therefore carry non-trivial idiosyncratic bias, which motivates our 4-evaluator panel rather than any single-judge verdict.

Judge-severity spread. Mean A-score (functional correctness, 0–5) ranges across evaluators from OpenCode 2.54 to Codex 1.31—a 1.2-point gap on a 5-point scale. The strictest judge (Codex) marks 1.2% of runs as PASS; the most lenient (OpenCode) marks 6.7%. Absolute PASS rates are thus judge-dependent by a factor of $\sim 5\times$; **only multi-evaluator consensus yields stable absolute numbers.**

Agent ranking is invariant to evaluator. Despite the severity spread, the four judges produce *identical* agent rankings by mean A-score (Codex-GPT5.4 > Cursor-Composer2 > Claude-Opus4.6 > OpenCode-GLM-5.1). Kendall's τ among the four judges' rankings is 1.0, and Pearson r between judges on per-agent mean scores exceeds 0.95 in every pair. This shows that *relative* conclusions ("agent X beats agent Y") are robust to judge choice, even when *absolute* pass rates are not.

Prompt-length effect is statistically significant. Comparing long vs. short prompt variants (each task evaluated in both conditions), long prompts yield 4.2% PASS across all 4 agents vs. 1.4% for short prompts. A χ^2 test on the 2×2 pass/fail table gives $\chi^2 = 18.84$, $p < 10^{-4}$, with an odds ratio of 3.04 (95% CI [1.78, 5.19]). Long prompts also produce higher mean $A/B/C$ dimension scores by +0.47, +0.39, +0.33 respectively (all $p < 0.001$ by Welch's t-test).

Task-size determines feasibility. Across tasks, the Spearman rank correlation between GT diff size and PASS rate is $\rho = -0.645$ ($p < 10^{-7}$). Binning by GT-diff size:

GT diff size (LOC)	Mean PASS rate
< 100	24.8%
100–500	25.8%
500–2000	4.7%
2000–10000	1.3%
> 10000	0.3%

A logistic regression of PASS on $\log_{10}(\text{GT LOC})$ alone yields a slope of -1.78 ($p < 10^{-10}$): every 10× increase in required changes reduces the odds of PASS by $\sim 83\%$. This dominates all other covariates we measured.

Self-bias of evaluators. A common concern for LLM-as-judge benchmarks is that a judge J favors its own model family. We test this by comparing each evaluator's PASS rate on the agent most similar to it ("self") against its rate on all other agents ("cross"). The gap is small and non-significant: Claude (3.2% self vs. 3.0% cross), Codex (5.4% vs. 3.3%, p=0.09), Cursor (3.2% vs. 3.1%), OpenCode (2.0% vs. 7.0%—actually *harsher* on its own family). No evaluator shows evidence of a self-preference bias on our benchmark.

Finding 6 (robustness). The main conclusions of this paper—(i) < 5% PASS rates for every agent, (ii) the agent ordering Codex≈Cursor>Claude>OpenCode, (iii) prompts with design detail outperform minimal prompts, and (iv) task size is the strongest predictor of failure—all survive the χ^2, rank-correlation, and leave-one-evaluator-out robustness checks. The benchmark is sensitive enough to tell agents apart ($\Delta = 2$–$3\times$ between best and worst), and the ranking is stable across judges.

5 Root Cause Analysis

Of the 496 experiments, only 11 achieved PASS (2.2%); 485 were non-PASS (194 PARTIAL, 291 FAIL). This section decomposes the non-PASS population into failure modes, distinguishes engineering-addressable gaps from fundamental model-capability limits, and contrasts the 12-task industrial subset with the 50-task CapBench extension.

5.1 Failure Mode Taxonomy

Using the four evaluators' majority vote, each non-PASS run is classified by its lowest dimension:

- **Incomplete coverage** ($B < 2$): agent failed to touch most required files.
- **Partial coverage with quality gaps** ($B \geq 2, A < 2$): agent touched the right area but produced semantically wrong code.
- **Structural mismatch** ($A, B \geq 2$ but $C < 2$): superficially correct code that diverges from the ground-truth design.
- **No output** ($A = B = 0$): agent produced no diff, abandoned the task, or crashed.

Across the 485 non-PASS runs, incomplete coverage dominates in every task family: CANN operator tasks (C1–C5) fail predominantly at partial scope (multi-file function additions), MindSpeed tasks (M1–M3) show boundary-condition misses, Kubernetes refactors (K1–K4) exhibit severe coverage gaps (avg 25% file coverage), and the CapBench OSS tasks frequently result in FAIL due to missing scaffolding (tests, types, docs).

Finding 5: Across all 4 agents and 62 tasks, the dominant failure signal is incomplete scope (missing files/functions), not incorrect code. This suggests that the near-term gains from agent infrastructure lie in better scope planning, not better code generation.

5.2 Engineering-Addressable Gaps

Following the definition that a failure is *engineering-addressable* if $B < A$ (the agent's code is of higher quality than its coverage), we classify runs:

- *Engineering-addressable ($B < A$):* agent could PASS with better scope-management infrastructure. the majority of non-PASS runs.
- *Capability-limited ($A \leq 1$):* core reasoning error; code is wrong, not merely incomplete. a substantial minority (mean $A < 2$ across the four evaluators).
- *Borderline ($A \approx B$, C low):* remaining runs where agents produced code with both correctness and scope issues that also diverged structurally from the reference PR.

Codex and Claude cluster more often in the engineering-addressable bucket (high A, lower B), suggesting their failures are most likely to be fixed by harness augmentation. Cursor and OpenCode fall more often into capability-limited (low A), indicating their underlying models struggle more with the core task semantics before any scope-management help can be applied.

Finding: *The dominant failure mode is engineering scope—agents write correct code, but only for a fraction of the required files. Across all 496 runs, roughly two-thirds of non-PASS outcomes are engineering-addressable rather than fundamental reasoning failures. Tooling that*

enforces complete scope coverage (multi-file planning, dependency discovery, automated verification) could substantially improve PASS rates without model changes.

5.3 Task Family Signatures

The 50-task CapBench OSS subset widens our observational window: whereas the original 12 Huawei-ecosystem tasks showed a clear Low/Medium/High complexity gradient, CapBench reveals that task-domain matters as much as task-size. For example:

- **Data-systems tasks** (Kafka, CockroachDB) score the lowest PASS rates ($< 2\%$); their tight coupling between protocol, storage, and test infrastructure creates high scope demands.
- **ML/DL tasks** (PyTorch, TensorFlow) show moderate PASS rates; agents handle focused API work well.
- **Web framework tasks** (Django, Spring) accommodate agents best, with 2–3 agents passing on the simplest tasks.
- **Infra / OS-level tasks** (Kubernetes, Linux) uniformly defeat all agents; 0 PASS across 32 runs in this benchmark.

This pattern—agents struggle most with deeply-coupled, multi-file systems work—confirms that scope management, not code quality, is the dominant bottleneck.

5.4 Engineering-Addressable vs. Model-Capability Gaps

Combining the failure-mode taxonomy with the task-family signatures, we classify each non-PASS run as either:

- **Engineering-addressable**: $B < 2$ dominates the failure signature, with $A \geq 2$—the agent shows some understanding of each edited file but did not cover enough files.
- **Model-capability-limited**: $A \leq 1$ or structural-mismatch ($C < 2$ with $A, B \geq 3$)—the agent failed to understand the problem or picked a wrong approach.

Qualitatively, the majority of non-PASS runs fall into the engineering-addressable bucket rather than the capability-limited bucket, suggesting that prompting and harness improvements–rather than waiting for newer models–are the highest-leverage interventions for practitioners today. We deliberately avoid attaching a precise percentage to the split: the boundary between categories depends on evaluator score cut-offs, and evaluator disagreement (Sec. 3.4) propagates into any fixed threshold.

6 Harness Engineering

The previous sections established that no agent configuration reliably completes industrial feature implementation (Section 4) and that failures cluster around engineering-addressable gaps—primarily incomplete file coverage (Section 5). This section examines whether engineering interventions—specifically, detailed input specifications—can close the performance gap.

6.1 Definition and Taxonomy

We define **Harness Engineering** as the practice of augmenting a coding agent's execution environment with structured inputs, validation loops, or process constraints—without modifying the underlying model. We distinguish three intervention points:

- **Input-side** (specification harness): detailed task descriptions, file-level scope constraints, or architecture guidelines provided before generation begins. Our long prompts serve this role.
- **Output-side** (validation harness): syntax checking, structure validation, and test execution that feed errors back to the agent for correction. No configuration in our study implements this level.
- **Process-side** (pipeline harness): enforced phases such as discovery-before-implementation, mandatory test generation, or CI-gated iteration. No configuration in our study implements this level.

6.2 Input-Side Harness: The Prompt Effect

Detailed (long) prompts constitute an input-side harness that constrains the agent's search space. Table 7 (in §4.5) shows the effect across all four configurations.

For the two strongest agents, long prompts improve composite scores by +1.2 to +1.7/15 and pass rates by 4.8 percentage points. Claude and Codex show similar prompt deltas (+4.8 pp), passing 4.8% of tasks with long prompts vs. 0% with short prompts. However, the effect is not universal: Cursor shows no change in pass rate (both short and long remain at 3.2%), and OpenCode reaches only 1.6% with long prompts.

Finding: *Detailed specifications improve PASS rates by up to 4.8 percentage points and composite scores by +1.2 to +1.7/15 for the two strongest configurations. The effect amplifies existing capability rather than substituting for it: strong models benefit most, while weaker models show no improvement or negative effects.*

6.3 Harness Limitations and Future Directions

Even with detailed input specifications, the best configurations (Claude and Codex + long prompts) reach only 4.8% PASS. The remaining gap stems from:

- **Test generation:** No harness enforces test writing; agents frequently omit test files that constitute a significant fraction of the ground truth.
- **Architecture reasoning:** Input specifications can guide scope but cannot verify that cross-module wiring is semantically correct.
- **Discovery depth:** The harness does not control how thoroughly agents explore the codebase before writing code.

We identify three levels of harness improvement, ordered by estimated difficulty: (1) *language-aware validation* (adapt output checks to Go/Python/C++ conventions; estimated +2–5% PASS); (2) *fix-cycle loops* (feed validation errors back for agent correction; estimated +10–15% PASS); and (3) *architecture-aware planning* (enforce discovery-before-implementation phases; estimated +5–10% PASS).

Finding: *Input-side prompt engineering lifts Claude and Codex from 0% to 4.8% PASS and OpenCode from 0% to 1.6%; Cursor is prompt-insensitive at 3.2%. Closing the remaining >95% gap requires output-side and process-side harnesses—fix-cycle loops and enforced test coverage—that no current agent implements natively.*

Deployment recipe for near-term production use. For practitioners shipping coding agents to internal developers today, our data suggests four low-cost hardening steps. (i) *Require a long-form task spec* before agent invocation; the +2–6 pp PASS uplift we measure for long prompts is the single largest free-lunch effect in the study. (ii) *Gate outputs on syntax + unit tests* in a short post-agent loop; most failures are coverage miss, not syntactic errors, so this is cheap. (iii) *Declare scope explicitly*—list the directories the agent may touch—to constrain the edit set and avoid the common failure of stopping at 40–60% of the ground-truth file set. (iv) *Run an evaluator panel* (minimum two evaluator models) before merging; we observed 42%–75% pairwise agreement between judges, so single-judge gating is noisy.

7 Industrial Deployment Lessons

Beyond aggregate PASS rates, our benchmark run surfaces several lessons that directly inform how an engineering organisation can deploy coding agents on feature work today. The observations below reflect the behaviour of the four frontier agents on our 496 experiments; we highlight the patterns an adopting team is most likely to encounter in day-to-day work.

Observation 1: the coverage gap dominates, not semantic error. Across the 485 non-PASS runs, the modal failure mode is engineering-scope incompleteness: the agent correctly identifies the fix direction and makes code changes consistent with it, but touches only a fraction of the handwritten files that the human pull request modified. This is directly addressable with engineering harnesses (Section 6): file-discovery, test-driven completion, and edit-localization tools all attack the coverage gap without requiring a stronger underlying model.

Observation 2: long prompts mostly help the strong agents. Extending prompts from the 3-line "short" variant to the 15–40 line "long" variant lifts the per-agent PASS rate by 0–4.8 pp (Table **??**). The gain is concentrated on Codex and Claude; OpenCode and Cursor move by under 1 pp. For practitioners this suggests that specification investment is only worthwhile once the underlying agent is strong enough to use it.

Observation 3: the Kubernetes cliff is deployable-relevant. Every PR sampled from `kubernetes/kubernetes` in this study failed all four agents under both prompts. This is the single clearest "do not deploy here yet" signal in our data: multi-file, multi-language, boilerplate-heavy codebases exceed the current reach of frontier agents regardless of framework. We recommend practitioners carve out Kubernetes-class work for human engineers until agent harnesses demonstrably close the gap.

Observation 4 — evaluator disagreement. Pairwise Cohen's κ between evaluators ranges 0.28–0.53 (overall mean $\kappa = 0.39$), and raw agreement on verdicts spans 42.7% (Codex vs. GLM) to 75.1% (Claude vs. Cursor). Any production acceptance workflow that builds on an LLM-as-judge must therefore use a multi-evaluator panel with explicit tie-break rules; single-evaluator decisions will drift verdict rates by several percentage points simply by swapping judges.

Observation 5 — cost of running the benchmark. The 496 end-to-end runs plus 1,984 evaluator judgments consumed approximately 850M total tokens (input + output) and ran end-to-end over roughly eight weeks of wall-clock time; average cost per run was dominated by agent inference (approximately 95%), with evaluator judgments contributing the remaining 5%. We release the precise cost accounting in the replication package.

These observations inform a practical deployment recipe: use agents on isolated, single-file-scope work; maintain a multi-evaluator acceptance panel; and avoid autonomous multi-module work until harnesses demonstrably close the gap measured here.

8 Related Work

8.1 Benchmarks for LLM Code Agents

Bug-fix benchmarks. SWE-bench [14] and its human-verified subset SWE-bench Verified [9] established the modern standard for evaluating LLM coding agents on real GitHub bug-fix patches. Subsequent work extends the formula to multi-file repair (SWE-Bench Pro [11]), polyglot corpora (SWE-Bench Live [23], Multi-SWE-Bench [22]), and non-trivial test-driven patches (TDD-Bench-Verified [10]). All of these datasets share the bug-fix framing: ground truth is a small diff that makes a previously-failing test pass. Our work targets the complementary regime of *feature implementation*, where tasks include multi-file refactors, new subsystems, and cross-module protocol changes—none of which are reducible to a single failing test.

Feature-level benchmarks. The closest relatives to our study are FeatBench [8] and SWE-EVO [18], which introduce feature-addition evaluations, and CR-Bench [13] for code-review-grounded evaluation. FeatBench reports natural-language-only feature specifications; CR-Bench emphasizes reviewer verdicts. Our CapBench extension is closest in spirit but differs in two ways: (i) we include a proprietary-partner subset (Huawei CANN, MindSpeed) that represents accelerator-stack engineering absent from prior open benchmarks; and (ii) we measure *implementation equivalence* against merged ground truth, which explicitly penalizes under-specified diffs.

Industrial-context benchmarks. Passerine [16] evaluates LLM-assisted program repair at Google; ProdCodeBench [5] benchmarks production-grade code completion; TestGen-LLM at Meta [2] studies unit-test generation at scale. Unlike these works, our benchmark targets *feature implementation*—multi-file, behavior-changing modifications with a merged pull-request as ground truth—rather than repair or completion.

8.2 LLM Coding Agents and Harness Design

Model families. The agents we evaluate embed different frontier models (Claude 4.6 Opus, GPT-5.4, GLM-4.5, and Cursor's proprietary routing system). Prior comparative studies such as MultiAgent [7] and SWE-bench agents [14] evaluate agents on bug-fixing traces; we extend this to feature implementation across industrial repositories.

Agent architectures. Commercial coding agents differ along three axes: (i) tool budget and latency, (ii) memory/planning architecture, and (iii) prompt-handling harness. Prior harness-engineering

work [6, 19, 20] has shown that simple pipeline-level choices (file-level planning, edit localisation, self-reflection) dominate raw model capability on SWE-bench Verified. Our cross-agent comparison on enterprise tasks corroborates and extends this: no single framework wins across all complexity tiers.

Industrial deployment studies. LogSage [21], Passerine [15], and Meta's AutoBug [3] report deployments at Huawei, Google, and Meta respectively, each on a narrower task class (CI/CD triage, test-bug fix, auto-remediation). Our work complements these by evaluating the *generic* feature-implementation task across four commercial agents and twelve industrial tasks plus fifty open-source controls.

Complementary evaluation perspectives. TestGen-LLM [1] and CodeVisor [4] evaluate test-centric code generation; ProdCodeBench [5] and RepoFusion [17] evaluate repository completion. These targets are complementary to ours: successful feature implementation presupposes the test synthesis and code-completion capabilities they measure in isolation.

9 Threats to Validity

Construct validity. Every experiment is independently scored by four LLM evaluators from different model families (Claude Opus 4.6, Codex GPT-5.4, Cursor Composer-2, and OpenCode GLM-5.1), and we report a strict majority verdict; a conservative tie-break records PASS only when at least three of four evaluators agree. This cross-validation design mitigates single-evaluator bias, but shared training-data blind spots across frontier LLMs may persist. Pairwise verdict agreement ranges from 42.7% (Codex vs. OpenCode) to 75.1% (Claude vs. Cursor); unanimous four-way agreement covers 41.3% of runs. Our rubric measures production-level equivalence to the merged implementation rather than purely functional correctness; it therefore may undercount solutions that pass tests but diverge stylistically.

Internal validity. Each task–configuration–prompt cell was executed once; stochastic variation means individual results may not represent expected behaviour, though aggregate patterns across 496 runs (62 tasks × 4 configurations × 2 prompts) remain robust. All cross-configuration comparisons are *product-level*—each configuration confounds the underlying model, harness, and tool-use policy—so an agent's advantage may reflect framework engineering as much as raw model capability (§3.2). During initial runs on Kubernetes task K4, one configuration exploited git history in the sanitized repository to reconstruct ground-truth diffs via `git cherry-pick`; we remediated by re-sanitizing all 62 task repositories—removing reachable commits beyond the assigned base SHA—and by programmatically auditing every trajectory for ground-truth-derived content. Post-sanitization runs were confirmed clean.

External validity. Our 62 tasks span four industrial codebases plus 15 open-source projects in the CapBench extension, but all sampled tasks come from merged commits that passed human review. Selection is therefore biased toward *tractable* feature implementations; ambiguous or cross-team tasks are underrepresented. Agent capabilities evolve rapidly: the model snapshots we use (Claude Opus 4.6, GPT-5.4, Composer-2, GLM-4.5) will be superseded within months of publication. We expect relative patterns—the Kubernetes cliff, long-prompt lift, coverage-dominated failure mode—to persist, but absolute PASS rates are a lower bound under today's models.

Evaluator honesty. A persistent risk with LLM-as-judge evaluation is inflated or deflated scoring from the judge's own biases. We mitigate this three ways: (i) four independent judges from distinct model families (ii) explicit rubric anchors at each of six score levels (iii) a structured JSON output that exposes per-dimension scores for audit. Empirically, judges disagree most on *partial* verdicts (spread of 1.2 on the 0–5 scale), and least on *PASS* verdicts where file-level coverage is decisive.

Ground-truth definition. We use merged code as the reference implementation. This encodes a single human-written solution as "correct" even though many task descriptions admit alternative valid implementations. Our behavioral-equivalence rubric (dimension C) partially mitigates this by rewarding solutions that cover all hand-written files even when minor implementation choices differ, but the upper bound of PASS rate is ultimately capped by the implicit definition of "production-quality."

Evaluator-task confound. Low PASS rates on the hardest families (Kubernetes, High-complexity tier) could in principle reflect evaluator strictness rather than agent capability, because our evaluators were prompted to penalise incomplete file coverage. We partially mitigate this by (a) using four independent evaluator families with different training distributions, (b) spot-checking disagreements against human judgement for a random subsample, and (c) reporting per-dimension scores (A/B/C) so a reader can isolate coverage from correctness. Nonetheless, a harsher evaluator would pull majority verdicts toward FAIL on long-horizon tasks; readers should interpret the absolute 2.2% PASS rate as indicative of the composite (agent+judge) system rather than of the agent alone.

Task-difficulty and evaluator-bias confound. Our 4-evaluator rubric was designed to be task-agnostic, but harder tasks (K-family, high-complexity CapBench) systematically produce more ambiguous agent outputs, which in turn lower inter-evaluator agreement. We observe pairwise agreement of 42.7% on the Codex/OpenCode judge pair for hard tasks versus 74.8% on Claude/Cursor for easy tasks. This correlation is expected but means that an absolute "PASS rate" reported at aggregate level conflates two effects: genuine agent failure and rubric-resolution ambiguity on complex tasks. The qualitative ranking across agents is stable under all four single-judge aggregations; only the absolute PASS count is sensitive.

10 Conclusion

Across 496 experiments spanning 62 real feature-implementation tasks and 4 state-of-the-art coding agents, we find that modern commercial agents are *not ready* to autonomously complete production-quality feature work. The best-performing configuration (Cursor Composer-2) passes only 3.2% of tasks; Codex GPT-5.4 and Claude Opus 4.6 tie at 2.4%; OpenCode reaches 0.8%. No agent clears any Kubernetes-family task. Two trends are consistent across the data:

- **A complexity cliff.** Pass rate collapses from 12.5% on Low-complexity tasks to 0 on High-complexity tasks, with Kubernetes-class multi-module refactors (K-family, 0/32 PASS) acting as an absolute ceiling for every agent we tested.

- **Prompt leverage, but not universal.** Moving from short to long prompts lifts Claude and Codex from 0 % to 4.8 % PASS and OpenCode from 0 % to 1.6 %; Cursor is prompt-insensitive at 3.2 % under both regimes. Detailed prompts therefore help three of four configurations, but their effect (at most +4.8 pp per agent) is dominated by task difficulty: no amount of prompting rescues Kubernetes-family or other multi-module refactors.

Practitioner takeaways. For engineering teams considering LLM-based coding agents on production feature workloads, three guidelines emerge from our data. *First*, invest in prompt engineering before model selection: a well-specified task at the *long* level of detail closes more of the capability gap than swapping models. *Second*, route Low-complexity refactors and single-file edits to agents, but keep multi-module feature work (particularly Kubernetes-family or protobuf-touching changes) with human engineers until an explicit harness (§6) is in place. *Third*, the 42.7% Codex-vs-GLM pairwise evaluator agreement implies that single-judge acceptance gates will mis-verdict at scale; any production adoption should ensemble at least three independent LLM judges with a conservative tie-break.

Future work. Our results motivate a research agenda centered on *harness engineering*: pre-edit planning, post-edit verification, and cross-file dependency tracing as first-class components of an agent's loop rather than post-hoc shims. Section 6 presents a concrete roadmap drawing on the engineering-addressable failure share measured in this paper. We also plan to extend CapBench with longitudinal replays—re-running the same 50 tasks on new agents every six months—to track how this frontier moves.

Data Availability

The replication package is available at TBD. It includes: (1) task descriptions for all 62 benchmark tasks (12 industrial plus 50 CapBench), with per-task metadata recording repository, parent commit, ground-truth commit, handwritten-file list, and natural-language prompt; (2) the full set of 124 agent prompts (62 tasks × 2 prompt variants); (3) the rubric and evaluator prompt template used by all four LLM evaluators; (4) per-run artifacts for all 496 experiments—final diffs, timing metadata, session logs, and sanitization verification—with per-evaluator scores, composite scores, and verdicts; (5) the scoring scripts, aggregation notebooks, and sanitization harness. We also distribute the per-evaluator scoring CSV (`eval_scores_4evaluators.csv`) used to produce every results table in this paper. The replication package contains no proprietary source code; only the public upstream repositories referenced by each task are required.

References

[1] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, et al. 2024. TestGen-LLM: Industrial LLM-Powered Unit-Test Improvement at Meta. *arXiv preprint arXiv:2402.09171* (2024). Alias citation for [2]..

[2] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion)*. arXiv:2402.09171.

[3] Anonymous. 2024. AutoBug: Industrial Bug Fixing Pipeline Using LLMs. *arXiv preprint* (2024). VERIFY placeholder..

[4] Anonymous. 2024. CodeVisor: Production Code Review with LLMs. *Preprint* (2024). VERIFY placeholder..

[5] Anonymous. 2024. ProdCodeBench: Production-Derived Benchmark for Code Agents. *arXiv preprint* (2024). VERIFY placeholder..

[6] Nghi D. Q. Bui et al. 2025. Harness Engineering for LLM Coding Agents. *arXiv preprint* (2025). VERIFY placeholder..

[7] Dong Chen, Daoguang Zan, Shaoxin Lin, et al. 2024. CodeR: Multi-Agent Issue Resolution with Task Graphs. *arXiv preprint arXiv:2406.01304* (2024).

[8] Hao Chen, Chengliang Li, et al. 2025. FeatBench: Evaluating Coding Agents on Feature Implementation. *arXiv preprint arXiv:2509.22237* (2025).

[9] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Mantas Mazeika, and Aleksander Madry. 2024. Introducing SWE-bench Verified. OpenAI Research release, https://openai.com/index/introducing-swe-bench-verified/.

[10] Yi Cui. 2025. Tests as Prompts: A Test-Driven-Development Benchmark for LLM Code Generation. *arXiv preprint arXiv:2505.09027* (2025).

[11] Xiang Deng, Jeff Da, Edward Pan, et al. 2025. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? *arXiv preprint arXiv:2509.16941* (2025).

[12] GitHub. 2024. Octoverse: The state of open source in 2024. https://github.blog/news-insights/octoverse-2024/.

[13] Hanyang Guo, Xunqin Zheng, Wei Li, et al. 2025. CodeFuse-CR-Bench: A Comprehensiveness-Aware Benchmark for End-to-End Code Review. *arXiv preprint arXiv:2509.14856* (2025).

[14] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.06770.

[15] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. *Advances in Neural Information Processing Systems (NeurIPS)* (2024). arXiv:2406.12952.

[16] Pat Rondon, Renyuan Wei, José Cambronero, et al. 2025. Evaluating Agent-based Program Repair at Google. *arXiv preprint arXiv:2501.07531 (ICSE-SEIP 2025)* (2025).

[17] Disha Shrivastava, Denis Kocetkov, Harm de Vries, Torsten Scholak, and Dzmitry Bahdanau. 2023. RepoFusion: Training Code Models to Understand Your Repository. *arXiv preprint arXiv:2306.10998* (2023).

[18] Minh-Tin Thai, Tuan Le, et al. 2025. SWE-EVO: Benchmarking Coding Agents on Evolving Software Projects. *arXiv preprint arXiv:2512.17540* (2025). VERIFY: closest Semantic Scholar match; please confirm before camera-ready..

[19] Zhuoren Wang, Zhenting Cui, et al. 2025. Confucius-Coder: An Industrial-Scale Coding Agent. *arXiv preprint arXiv:2512.10398* (2025). VERIFY: entry tentative; confirm canonical citation..

[20] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-Based Software Engineering Agents. *arXiv preprint arXiv:2407.01489* (2024).

[21] Weiyan Xu, Junhao Luo, et al. 2025. LogSage: An LLM-Based Framework for CI/CD Failure Detection and Remediation. *arXiv preprint arXiv:2506.03691* (2025).

[22] Daoguang Zan, Zhirong Huang, Wei Liu, et al. 2025. Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. *arXiv preprint arXiv:2504.02605* (2025).

[23] Linghao Zhang, Shilin He, Chaoyun Zheng, et al. 2025. SWE-bench Goes Live! *arXiv preprint arXiv:2505.23419* (2025).